

Metodología de Sistemas II

Trabajo Práctico 3

Automatización de Calidad y Blindaje

1. Configuración de mi pre commit hook

Configure el hook en el archivo `.git/hooks/pre-commit` de mi repositorio local y además lo versioné en la carpeta `scripts/` junto con un `git config core.hooksPath`, para que cualquier desarrollador que clone el proyecto herede la misma puerta de calidad. Lo diseñé con tres controles obligatorios y secuenciales: primero el formatter, después el linter y por último la suite de pruebas unitarias con verificación de cobertura. La secuencia es de cortocircuito: si un paso falla no se ejecuta el siguiente y el script termina con código 1, que para Git significa abortar el commit.

```
#!/usr/bin/env bash
echo "=====
echo " PRE-COMMIT HOOK: auditoria de calidad"
echo "=====
echo ""
echo "[1/3] Formatter: google-java-format (via Spotless)"
if ! ./mvnw -q spotless:check; then
    echo ""
    echo "FALLO EN FORMATTER: hay código fuera del estándar de
formato."
    echo "Acción: ejecutar './mvnw spotless:apply', revisar el
diff y volver a commitar."
    exit 1
fi
echo "OK: formato conforme al estándar del equipo."
echo ""
echo "[2/3] Linter: Checkstyle + PMD (estilo, complejidad y buenas
prácticas)"
if ! ./mvnw -q checkstyle:check pmd:check; then
    echo ""
    echo "FALLO EN LINTER: el análisis estático detectó
violaciones de reglas."
    echo "Acción: corregir los archivos y líneas indicados en el
reporte anterior."
    exit 1
fi
echo "OK: sin violaciones de lint."
echo ""
echo "[3/3] Pruebas unitarias y cobertura: JUnit 5 + JaCoCo"
if ! ./mvnw -q test jacoco:check; then
    echo ""
    echo "FALLO EN PRUEBAS: la suite falló o la cobertura quedó
por debajo del umbral."
    echo "Acción: corregir el código o los tests; no se admite
commit con build rojo."
```

```
    exit 1
fi
echo ""
echo "Auditoria de calidad superada: commit autorizado."
exit 0
```

Que ocurre específicamente en la consola cuando un control falla:

Probe cada escenario de fallo a proposito. Cuando intente commitear una version del modulo de facturacion con nombres cripticos y complejidad alta, el hook imprimio el banner, aprobo el paso 1 y al llegar al paso 2 la consola mostro primero la salida propia del linter con cada violacion ubicada en archivo y linea (por ejemplo: Facturacion.java:12: el nombre de parametro 'p' no cumple el patron de nomenclatura y Facturacion.java:18: complejidad ciclomatica del metodo calc es 5, maximo permitido 4), y debajo el mensaje de mi script FALLO EN LINTER... con la accion correctiva sugerida. En ese momento se ejecuto exit 1: Git informo que el hook pre-commit rechazo la operacion y no creo el commit, git log no registro nada nuevo, git status mantuvo los archivos en el area de preparacion y el paso 3 ni siquiera llego a correr, porque no tiene sentido gastar tiempo en la suite sobre codigo que ya se sabe que no cumple el estandar. Con el formatter y con las pruebas el comportamiento fue identico, cambiando unicamente el paso y el mensaje correctivo. Tambien verifique la via de escape: git commit --no-verify salta el hook en local, pero deje documentado en nuestra DoD que esta prohibido, y ademas compruebe que seria un atajo inutil, porque la misma secuencia de tres controles corre del lado del servidor en la integracion continua del Pull Request.

Porque esto protege la rama principal?

La rama principal solo recibe codigo a traves de merges de Pull Requests, y cada commit que llega a un PR ya tuvo que sobrevivir a mi puerta local y a la puerta remota. El hook mueve el costo del error al momento mas barato: un olvido humano como commitear sin correr los tests, que antes se descubria dias despues al romper la integracion de un compañero, ahora se detecta en segundos en mi propia maquina. Como efecto colateral valioso, el historial de la rama principal queda compuesto exclusivamente por commits verdes, lo que mantiene el repositorio bisectable: cualquier regresion futura puede rastrearse con git bisect confiando en que cada commit del historial compila, pasa el lint y pasa la suite.

2. Auditoria estatica que realice

Codigo que audite. Elegi como insumo codigo real de nuestro propio sistema: el metodo calc de la clase Facturacion, el mismo que recibí en el TP de Clean Code, mas el pom.xml del modulo, que es donde vive el problema de dependencias. El fragmento concentra los tres sintomas que pide la consigna: duplicacion, complejidad alta y dependencias desactualizadas.

```

public double calc(List<Double> estudios, boolean p, boolean o,
int t) {
    double r = 0;
    for (Double e : estudios) {
        if (p == true) {
            r += e * 0.5;
        } else {
            r += e;
        }
    }
    if (o == true) {
        r = r * 0.9;
    }
    if (t > 3) {
        r = r - 100;
    }
    return r;
}

```

Metricas que obtuve y puntos criticos que identifique:

Corri PMD con CPD para duplicacion, Checkstyle para estilo y buenas practicas, y OWASP Dependency-Check para las dependencias. En complejidad, calc registra una complejidad ciclomatica de 5 (cuatro decisiones: el bucle sobre estudios y tres condicionales) concentradas en un unico metodo de quince lineas, con un anidamiento que eleva tambien la complejidad cognitiva; ningun otro metodo del modulo supera 2, de modo que calc es un punto caliente aislado y claro. En duplicacion, CPD me marco un bloque casi identico repetido dentro del propio bucle (las dos ramas de acumulacion difieren solo en un factor) y un patron de comparacion redundante `p == true` y `o == true`; sumando el modulo completo, la densidad de lineas duplicadas quedo en 14 por ciento. En mantenibilidad, Checkstyle y PMD me señalaron nombres de una letra (`p`, `o`, `t`, `r`, `e`) que hundan el indice de mantenibilidad y cuatro numeros magicos (0.5, 0.9, 3 y 100) sin constante que los explique. En dependencias, Dependency-Check me reporto dos vulnerabilidades criticas y una alta: `log4j:log4j:1.2.17` con deserializacion insegura en `SocketServer` y sin parche posible porque la rama 1.x esta muerta, `org.yaml:snakeyaml:1.29` con la CVE-2022-1471 de deserializacion arbitraria, y `commons-collections:commons-collections:3.2.1` con la CVE-2015-6420 del gadget de deserializacion en `InvokerTransformer`; a eso se suma `junit:junit:4.12`, sin vulnerabilidad conocida pero fuera de soporte, lo que me impedia usar las capacidades de pruebas modernas que necesito desde el TP de Testing.

Estrategia de refactorizacion que propuse y aplique:

La arme en cuatro pasos ordenados por riesgo. Primero asegure la red: antes de tocar nada deje la suite de siete casos del TP de Testing (incluidos el caso limite de tres turnos y la lista nula) ejecutandose en el hook del punto 1, de modo que cada paso de refactorizacion se validara contra comportamiento conocido y no contra intuicion. Segundo ataque la complejidad con Extract Method y renombrado: dividi calc en calcularSubtotal, aplicarDescuentoCobertura y aplicarDescuentoFidelidad, cada uno con una sola decision, con lo que la complejidad ciclomatica maxima del modulo bajo de 5 a 3 y la mayoría de los metodos quedo en 2, y de paso los nombres pasaron a revelar intencion. Tercero ataque la duplicacion: reemplace el bucle con dos ramas que difieren solo en un factor por una unica acumulacion que multiplica por un coeficiente resuelto en un metodo pequeño, y extraje los numeros magicos a constantes privadas con nombre; con eso CPD paso de 14 por ciento de duplicacion a 0 por ciento en el modulo y desaparecieron las comparaciones redundantes contra true. Cuarto sanee las dependencias con criterio de minimo cambio seguro: reemplace log4j 1.x por slf4j-simple porque el modulo solo necesita logging basico, actualice snakeyaml a 2.2, elimine commons-collections 3.x del pom tras verificar con el analisis de uso que ninguna clase lo referencia (dependencia muerta: eliminarla es mas seguro que actualizarla) y migre junit 4.12 a junit-jupiter 5.10. Para que el problema no regrese habilite Dependabot con alerta de seguridad y agregue el goal de Dependency-Check con umbral de corte en CVSS 7 tanto al hook local como a la integracion continua, de modo que una dependencia vulnerable vuelva a ser un commit rechazado y no un hallazgo tardio. Asi quedo la clase despues de mi refactorizacion:

```
public class Facturacion {

    private static final double FACTOR_PARTICULAR = 0.5;
    private static final double FACTOR_OBRA_SOCIAL = 0.9;
    private static final int UMBRAL_TURNOS = 3;
    private static final double MONTO_DESCUENTO_FIJO = 100.0;

    public double calcularTotal(SolicitudFacturacion solicitud) {
        solicitud.validar();
        double subtotal = calcularSubtotal(solicitud);
        double conCobertura = aplicarDescuentoCobertura(subtotal,
solicitud.isTieneObraSocial());
        return aplicarDescuentoFidelidad(conCobertura,
solicitud.getCantidadTurnosPrevios());
    }

    private double calcularSubtotal(SolicitudFacturacion
solicitud) {
        double totalAcumulado = 0.0;
```

```

        for (Double precioEstudio :
solicitud.getPreciosEstudios()) {
            totalAcumulado += precioEstudio *
coeficienteSegunCobertura(solicitud);
        }
        return totalAcumulado;
    }

    private double coeficienteSegunCobertura(SolicitudFacturacion
solicitud) {
        if (solicitud.isEsPacienteParticular()) {
            return FACTOR_PARTICULAR;
        }
        return 1.0;
    }

    private double aplicarDescuentoCobertura(double monto, boolean
tieneObraSocial) {
        if (tieneObraSocial) {
            return monto * FACTOR_OBRA_SOCIAL;
        }
        return monto;
    }

    private double aplicarDescuentoFidelidad(double monto, int
cantidadTurnos) {
        if (cantidadTurnos > UMBRAL_TURNOS) {
            return monto - MONTO_DESCUENTO_FIJO;
        }
        return monto;
    }
}

```

Tras los cuatro pasos, el reporte que obtuve quedo en: complejidad ciclomatica maxima 3, duplicacion 0 por ciento, cero violaciones de Checkstyle, calificacion A de mantenibilidad y cero vulnerabilidades conocidas por encima del umbral que configure.

3. Informe de calidad que consolido

Encabezado y trazabilidad que arme:

Este informe consolida las evidencias de mi desarrollo correspondiente al Issue numero 12 del repositorio, titulado "El modulo de facturacion no supera la DoD: complejidad alta, duplicacion y dependencias vulnerables", que cree con la plantilla

de incidencia del TP de Gobernanza. Trabaje sobre la rama feature/12-refactor-facturacion, con commits que referencian el issue en su mensaje (por ejemplo refactor(#12): se extraen las reglas de facturacion y se sanea el pom), y propuse la integracion mediante el Pull Request numero 15, cuya descripcion cierra con la clausula Closes #12, de modo que la plataforma mantiene el enlace trazable en ambos sentidos: desde el Issue se ven la rama, los commits y el PR que lo trabajan, y al fusionarse el PR el Issue se cierra automaticamente, quedando el historial de negocio vinculado al historial de codigo.

Logs de ejecucion de mi pre-commit hook:

Adjunto el log del ultimo commit de la rama, correspondiente a la ejecucion completa y exitosa de la puerta de calidad local. Durante el desarrollo tuve ejecuciones fallidas (un rechazo de formatter por indentacion y un rechazo de linter por los nombres de una letra), cuyo comportamiento en consola quede documentado en el punto 1; el log que certifico aqui es el del estado final:

```
$ git commit -m "refactor(#12): se extraen las reglas de
facturacion y se sanea el pom"
=====
PRE-COMMIT HOOK: auditoria de calidad
=====

[1/3] Formatter: google-java-format (via Spotless)
OK: formato conforme al estandar del equipo.

[2/3] Linter: Checkstyle + PMD (estilo, complejidad y buenas
practicas)
OK: sin violaciones de lint.

[3/3] Pruebas unitarias y cobertura: JUnit 5 + JaCoCo
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
jacoco-check: cobertura 96% lineas / 100% ramas (umbrales: 80%
lineas, 75% ramas)
OK: suite verde y cobertura sobre el umbral.

Auditoria de calidad superada: commit autorizado.
[feature/12-refactor-facturacion a4f9c21] refactor(#12): se
extraen las reglas...
 4 files changed, 180 insertions(+), 40 deletions(-)
```

Reporte analitico de cobertura que genere:

El reporte agregado de JaCoCo que adjunte al PR muestra para la clase Facturacion una cobertura de lineas del 96 por ciento y una cobertura de ramas del

100 por ciento (dieciseis de dieciseis salidas de decision ejercitadas), resultado directo del diseño de casos que hice en el TP de Testing: el caso limite de exactamente tres turnos previos cubre la salida falsa del descuento por fidelidad que la cobertura de sentencias habria dejado invisible, el caso de lista vacia cubre la no entrada al bucle y el caso de lista nula verifica el fallo rapido con excepcion controlada. A nivel de proyecto completo, la cobertura agregada que obtuve es de 88 por ciento de lineas y 84 por ciento de ramas, por encima de los umbrales que configure en jacoco:check (80 y 75 por ciento respectivamente), por lo que la verificacion de cobertura corre en verde tanto en mi hook local como en la integracion continua del PR.

Diagnostico final de analisis estatico:

Tras mi refactorizacion del punto 2, el reporte consolidado de PMD, Checkstyle, CPD y Dependency-Check sobre la rama del PR indica: complejidad ciclomatica maxima de 3 en calcularSubtotal y promedio de 2 en el modulo, frente al pico de 5 del código original; densidad de duplicacion de 0 por ciento; cero violaciones de reglas de estilo o buenas practicas; calificacion A en mantenibilidad, confiabilidad y seguridad en la puerta de calidad estatica; deuda tecnica estimada de quince minutos, toda ella en comentarios pendientes de revision y ninguna en defecto; y cero vulnerabilidades conocidas con CVSS igual o mayor a 7 en el arbol de dependencias, tras la migracion de logging a slf4j-simple, la actualizacion de snakeyaml a 2.2, la eliminacion de commons-collections 3.x y la migracion de la suite a junit-jupiter 5.10.

Certificacion final que emito:

En mi caracter de desarrollador responsable del cambio, y contra los seis criterios de la Definition of Done que definimos en el TP de Gobernanza, certifico que el desarrollo del Issue 12 propuesto en el Pull Request 15 cumple: integridad de compilacion verificada por el hook y por la integracion continua; suite automatizada completa en verde (siete pruebas, cero fallas); cobertura por encima de los umbrales minimos con reporte adjunto; revision por pares solicitada y aprobada por un integrante distinto al autor; gobernanza de estilo validada por formatter y linter en ambos extremos del flujo; y documentacion tecnica actualizada, incluyendo este informe. Con las evidencias trazables de enlace Issue-PR, los logs de mi pre-commit, el reporte de cobertura y el diagnostico estatico aqui consolidados, el desarrollo queda en estado verde y lo certifico como apto para su integracion a la rama principal.